

Practica - 11 Analítica Espacial: Arreglos y Tablas

Luis Gálvez

Tabla de contenidos

1	Proposito de la practica	1
1.1	Ejercicio 1: Vectorización y álgebra espacial (Haversine)	1
1.1.1	Desarrollo	1
1.2	Ejercicio 2: Limpieza, filtros y agregación tabular	2
1.2.1	Desarrollo	2

1 Proposito de la practica

1.1 Ejercicio 1: Vectorización y álgebra espacial (Haversine)

La **fórmula de Haversine** permite calcular la gran distancia sobre la esfera terrestre dadas dos coordenadas en latitud y longitud.

Tu misión es crear una función vectorizada (en Python, R o Julia) llamada **calcular_distancias** que reciba como argumento el `DataFrame` **df** de censos (que construimos en la sección 4) y calcule la distancia en kilómetros desde **Bogotá (Lat: 4.7110, Lon: -74.0721)** a todas las demás ciudades de la tabla, agregando el resultado en una nueva columna llamada **Distancia_Bogota**.

- *Tip: ¡Recicla tu código!* Recuerda que en el capítulo de *Funciones y Clases* ya construiste matemáticamente la función Haversine. Solo tienes que traerla a este script y adaptarla para que funcione directamente sobre vectores en lugar de números individuales, **evitando usar ciclos for**.
- *Pista de rendimiento:* Aplica la matemática directamente sobre las columnas completas del *DataFrame* (ej. `df['Latitud'] * np.pi / 180`).

1.1.1 Desarrollo

`DataFrame` original:

```
Ciudad  Latitud  Longitud
```

0	Bogotá	4.7110	-74.0721
1	Medellín	6.2442	-75.5812
2	Cali	3.4516	-76.5320
3	Barranquilla	10.9685	-74.7813

DataFrame con distancias:

	Ciudad	Latitud	Longitud	Distancia_Bogota
0	Bogotá	4.7110	-74.0721	0.000000
1	Medellín	6.2442	-75.5812	238.672963
2	Cali	3.4516	-76.5320	306.670166
3	Barranquilla	10.9685	-74.7813	700.169599

1.2 Ejercicio 2: Limpieza, filtros y agregación tabular

Para consolidar tus habilidades analíticas con *DataFrames*, crea un script que resuelva el siguiente flujo de trabajo simulado:

1. **Creación:** Genera un DataFrame con los siguientes datos de estaciones meteorológicas:
 - Estación: ["Páramo", "Valle", "Costa", "Selva", "Sabana"]
 - Elevación: [3200, 1500, 15, 200, 2600]
 - Precipitación: [850.5, 1200.0, None, 3000.5, None] (*Usa el tipo de nulo apropiado según tu lenguaje: `None/np.nan`, `NA` o `missing`*).
2. **Imputación:** Los sensores de la Costa y la Sabana fallaron. Rellena los datos de precipitación nulos utilizando el valor **promedio (media)** de las estaciones válidas.
3. **Filtro espacial (Slicing):** Extrae en un nuevo DataFrame llamado `estaciones_altas` solo aquellas filas cuya elevación sea estrictamente mayor a 1000 metros.
4. **Cálculo final:** Muestra en consola cuál es la precipitación promedio anual reportada *exclusivamente* en las estaciones altas.

1.2.1 Desarrollo

DataFrame original:

	Estación	Elevación	Precipitación
0	Páramo	3200	850.5
1	Valle	1500	1200.0
2	Costa	15	NaN
3	Selva	200	3000.5
4	Sabana	2600	NaN

DataFrame después de imputar valores nulos:

	Estación	Elevación	Precipitación
--	----------	-----------	---------------

0	Páramo	3200	850.500000
1	Valle	1500	1200.000000
2	Costa	15	1683.666667
3	Selva	200	3000.500000
4	Sabana	2600	1683.666667

Estaciones altas:

	Estación	Elevación	Precipitación
0	Páramo	3200	850.500000
1	Valle	1500	1200.000000
4	Sabana	2600	1683.666667

Precipitación promedio anual en estaciones altas:
1244.722222222224